

TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics

Software Engineering

OPERATING SYSTEMS

Laboratory Work No. 11

The Race and the Semaphore

Performed by:
Bianca-Andreea Tataroi
Group FAF-242

Checked by:
Patricia Reițman

Chisinau, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Part 1 – Observing a Race Condition **2**

3.1 Implementation and Execution 2

3.2 Results 3

3.3 Observations 3

4 Part 2 – Fixing the Race with Synchronization **3**

4.1 Mutex Implementation 3

4.2 Semaphore Implementation 4

4.3 Experiment: Semaphore Initialized to 2 4

4.4 Execution Time Comparison 5

4.5 Observations 6

5 Part 3 – A Semaphore as a Signal **6**

5.1 Correct Execution with Initial Value 0 6

5.2 Experiment with Initial Value 1 6

5.3 Observations 7

6 Part 4 – A Semaphore as a Counter: Parking Lot **7**

6.1 Parking Lot with Three Spaces 7

6.2 Experiment with One Space 7

6.3 Observations 8

7 Conclusion **8**

8 GitHub Repository **9**

1 Introduction

Concurrency allows multiple threads to make progress within the same process. When threads update a shared variable without synchronization, their operations can overlap and lead to a *race condition*. A program may appear to work in some executions while producing incorrect results in others.

In this laboratory, two threads increment a shared counter. The problem is first demonstrated without synchronization and then addressed using a mutex and a semaphore. Further experiments use semaphores to coordinate thread order and to limit the capacity of a simulated parking lot.

2 Objectives

The main objectives of this laboratory work are:

- Observe a race condition caused by concurrent updates to a shared counter.
- Protect a critical section using a pthread mutex.
- Use a semaphore with an initial value of 1 as a lock.
- Compare execution times of unsynchronized and synchronized programs.
- Use a semaphore initialized to 0 to signal when data is ready.
- Use a counting semaphore to manage limited parking spaces.

3 Part 1 – Observing a Race Condition

3.1 Implementation and Execution

The file `race.c` starts two threads. Each thread increments the same global variable `counter` one million times, using `counter++` without a lock. The expected total is 2,000,000.

The program was compiled and run using:

```
gcc -Wall -pthread -o race race.c
./race
./race
./race
```

3.2 Results

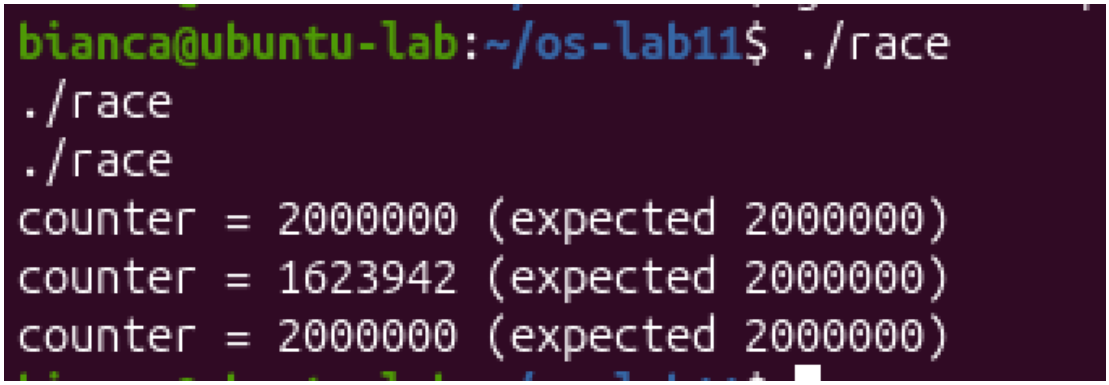


Figure 1: Three executions of the unsynchronized counter in race.c

The first and third runs produced 2,000,000, but the second produced only 1,623,942. Thus, 376,058 updates were missing in that execution.

3.3 Observations

Question 1: What three steps are involved in `counter++`?

Conceptually, the program reads the current value, adds 1, and writes the new value back.

Question 2: How can two threads lose one increment?

If both threads read the same old value, they may both calculate and write the same new value. Two increments have been attempted, but the shared counter increases only once.

Question 3: Why can the result change between runs?

The operating system schedules the threads differently, changing the timing and overlap of their memory accesses. In C, unsynchronized conflicting accesses also constitute a data race, so the program's behavior is not guaranteed.

4 Part 2 – Fixing the Race with Synchronization

4.1 Mutex Implementation

The `mutex.c` program protects the increment using `pthread_mutex_lock()` and `pthread_mutex_unlo` Only one thread at a time can execute the critical section.

```
gcc -Wall -pthread -o mutex mutex.c
./mutex
./mutex
./mutex
```

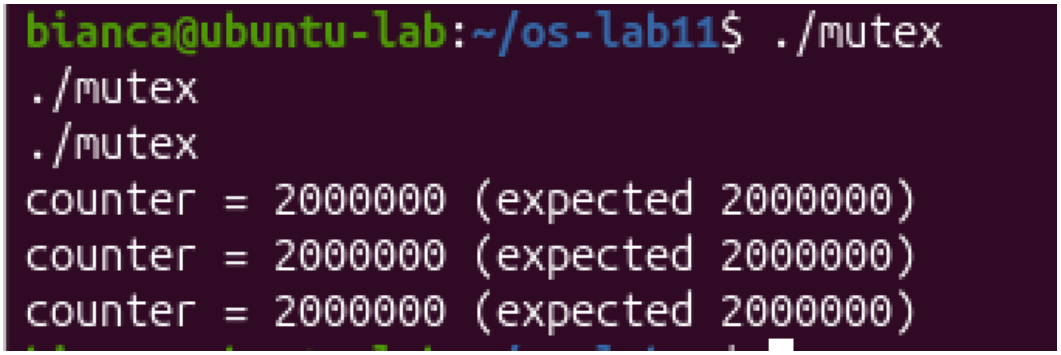


Figure 2: Three correct executions using a mutex

All three runs returned 2,000,000, as expected.

4.2 Semaphore Implementation

In `sem.c`, a semaphore is initialized with an initial value of 1. Each thread calls `sem_wait()` before incrementing the counter and `sem_post()` afterward.

```
gcc -Wall -pthread -o sem sem.c
./sem
./sem
./sem
```

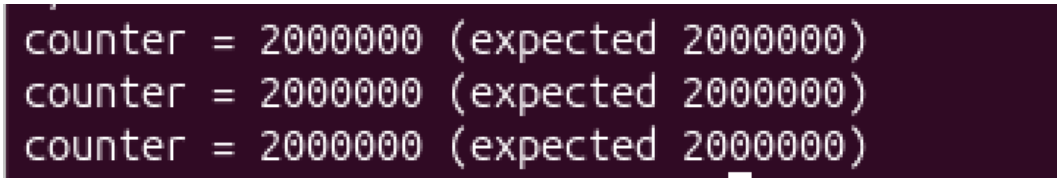


Figure 3: Correct counter values using a semaphore initialized to 1

The semaphore ensures mutual exclusion during the increment, and all three runs returned 2,000,000.

4.3 Experiment: Semaphore Initialized to 2

The initial value was temporarily changed from 1 to 2. This permits both threads to pass `sem_wait()` and access the counter concurrently, so the race returns.

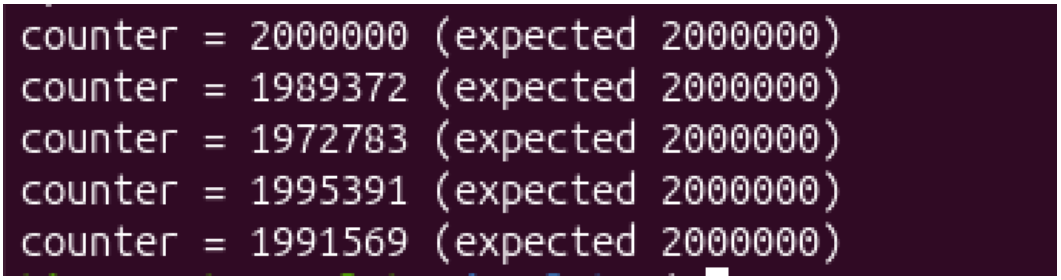


Figure 4: Incorrect counter values when the semaphore starts at 2

The screenshot shows several values smaller than 2,000,000, confirming that a semaphore initialized to 2 does not protect this critical section. After the experiment, the semaphore was restored to 1.

4.4 Execution Time Comparison

The following commands measured the elapsed time of each version:

```
time ./race
time ./mutex
time ./sem
```

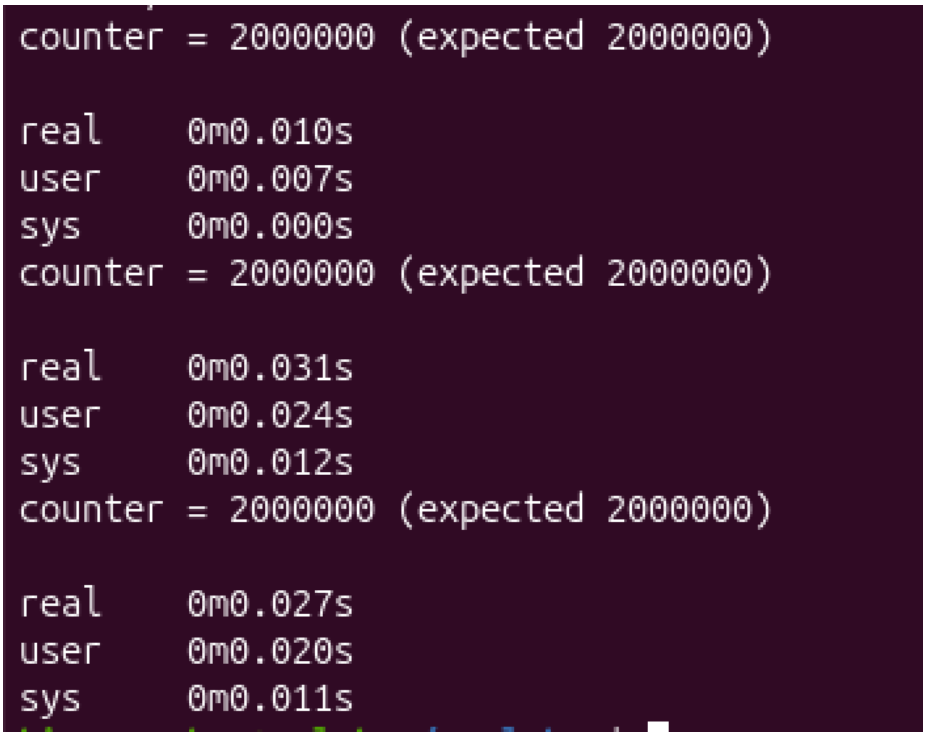


Figure 5: Execution times of race.c, mutex.c, and sem.c

Program	**Elapsed time (real)**
Race	0.010 s
Mutex	0.031 s
Semaphore	0.027 s

Table 1: Measured execution times on the Ubuntu virtual machine

The unsynchronized version was fastest in this measurement but is unreliable. Synchronization introduces overhead due to lock operations and possible thread contention. The semaphore happened to be slightly faster than the mutex in this run; this ordering is not universal.

4.5 Observations

Question 1: Do the mutex and semaphore versions always return 2,000,000?

Yes. Each of the three tested runs of both correctly synchronized versions returned the expected value.

Question 2: What happens with `sem_init(&s, 0, 2)`?

Two threads are allowed into the critical section at once, so mutual exclusion is lost and increments can be missed.

Question 3: Why are the synchronized versions slower?

Lock acquisition, release, and contention have a runtime cost. The overhead is visible in the measured elapsed times.

5 Part 3 – A Semaphore as a Signal

5.1 Correct Execution with Initial Value 0

The `order.c` program has a writer thread A and a reader thread B. B begins first and waits on a semaphore initialized to 0. After a one-second delay, A writes `hello` and calls `sem_post()`, allowing B to read the message.

```
gcc -Wall -pthread -o order order.c
./order
```

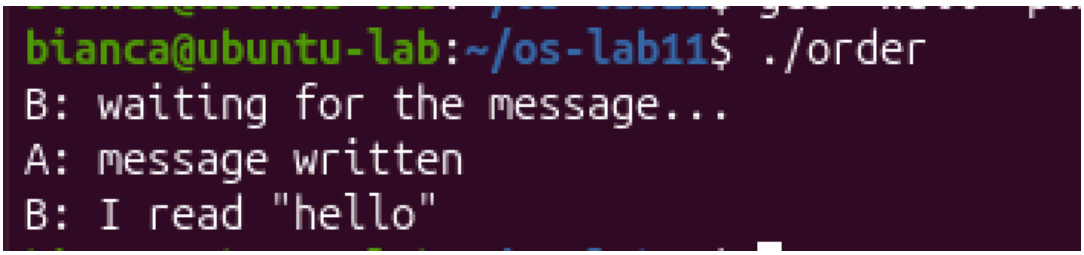


Figure 6: Reader waits until the writer signals that the message is ready

The output confirms that B read `hello` only after A wrote it.

5.2 Experiment with Initial Value 1

The initial value was temporarily set to 1. B could then pass `sem_wait()` without waiting for A.

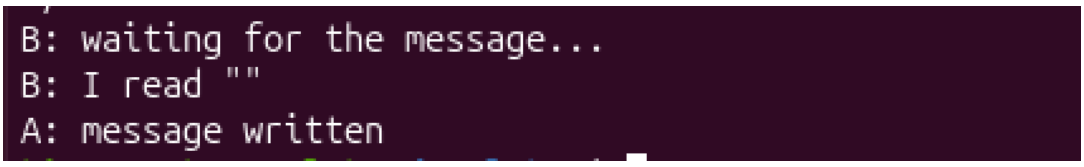


Figure 7: Reader accesses the empty message when the semaphore starts at 1

The reader printed an empty string before the writer announced that the message was written. The initial value was then restored to 0.

5.3 Observations

Question 1: What did B read when the semaphore started at 1, and why?

B read an empty string because the semaphore already permitted it to proceed before A had written the message.

Question 2: What does a semaphore initialized to 0 mean?

It represents an unavailable signal: the waiting thread blocks until another thread posts to the semaphore.

6 Part 4 – A Semaphore as a Counter: Parking Lot

6.1 Parking Lot with Three Spaces

The `parking.c` program simulates six cars as threads and three parking spaces. The semaphore tracks the number of available spaces, while a mutex protects the `inside` counter and corresponding printed messages.

```
gcc -Wall -pthread -o parking parking.c
./parking
```

```
car 1 enters (inside: 1)
car 2 enters (inside: 2)
car 6 enters (inside: 3)
car 6 leaves (inside: 2)
car 3 enters (inside: 3)
car 2 leaves (inside: 2)
car 1 leaves (inside: 1)
car 5 enters (inside: 2)
car 4 enters (inside: 3)
car 3 leaves (inside: 2)
car 4 leaves (inside: 1)
car 5 leaves (inside: 0)
```

Figure 8: Six cars using a parking lot with three available spaces

During the execution, the largest reported value of `inside` was 3. All six cars entered and left, and the final value was 0.

6.2 Experiment with One Space

The definition was changed temporarily from `PLACES 3` to `PLACES 1`.

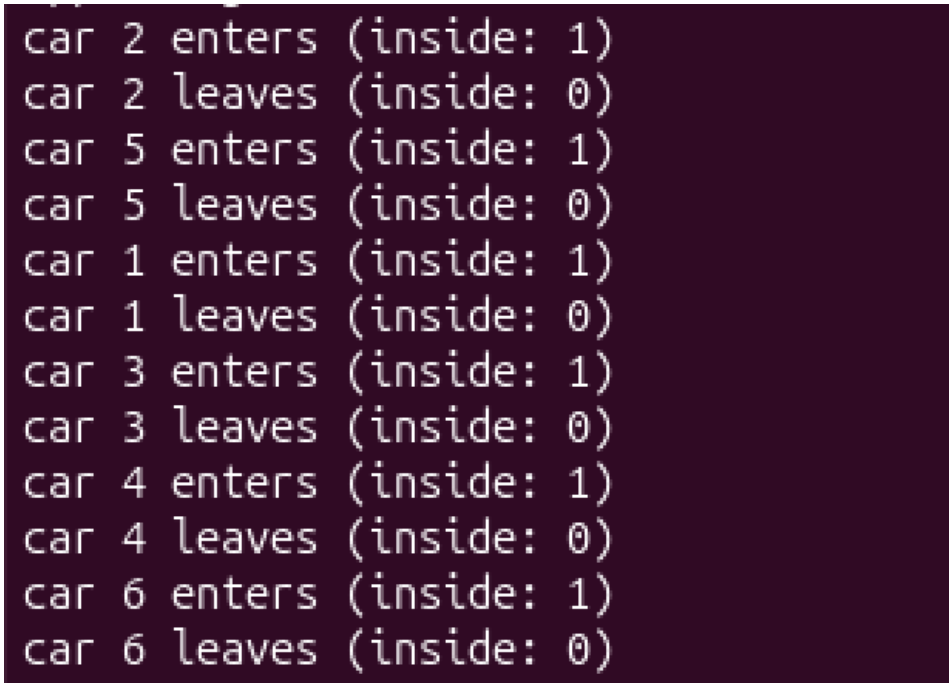


Figure 9: Parking experiment with a single available space

Cars entered and left one at a time, and `inside` never exceeded 1. The final value of `PLACES` was restored to 3.

6.3 Observations

Question 1: What is the largest value of `inside`, and which line guarantees the limit?

The largest value was 3. The call `sem_wait(&places)` reserves one of the three available semaphore units before a car enters, preventing a fourth simultaneous entry.

Question 2: What does the semaphore become when `PLACES` is 1?

It behaves as a binary semaphore, permitting at most one car in the parking lot at a time.

7 Conclusion

This laboratory demonstrated how unprotected access to shared memory can produce a race condition. The unsynchronized counter sometimes lost increments, whereas both the mutex and the semaphore initialized to 1 produced the expected result of 2,000,000 in all tested runs.

The additional experiments illustrated two other uses of semaphores: signaling when a message is ready and counting available resources in a parking lot. The results also showed that synchronization improves correctness at the cost of some execution overhead.

8 GitHub Repository

The laboratory source code and `report.md` are intended for submission in the following repository:

https://github.com/Biancatataroi/Operating-Systems-Lab11